NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS,PUNE
DEPARTMENT OF AIML ENGINEERING
ACADEMIC YEAR: 2025-26

Database Management Systems Lab Manual SE AIML Semester -II

2025 – 26

National Education Policy (NEP)-2020 Compliant Curriculum

Second Year Engineering (2024 Pattern) in

Artificial Intelligence and Machine Learning

(With effect from Academic Year 2025-26)

Prepared by:

Sri.A.A.Salunkhe
Assistant Professor

Sri.A.D.Deshmukh
Assistant Professor

Institute Vision & Mission

Vision:

The institute vision is to establish the centre for excellence in professional development and entrepreneurship development resulting into the enhancement of rural area. The institute also vision for developing professionals and citizens/citizenship to foster professional and rural development.

Mission:

NESGI offers skill based programs, graduate and post-graduate programmes in engineering and management to build competent manpower to suit the ever-changing requirements in industry and business by supporting students for continual development through excellence, technology based instructions and overall development of personality in all domains. The institute provides industry based education and practical training to the rural base.

Departmental Vision & Mission

Vision:

To become a leading center of excellence in Artificial Intelligence and Machine Learning education, innovation, and research, cultivating ethically driven professionals who pioneer AI technologies to address global challenges and contribute to sustainable development, with a special focus on empowering rural and underserved communities.

Mission:

- The AIML Department is dedicated to providing cutting-edge education and fostering innovation in Artificial Intelligence and Machine Learning.
- Our mission is to equip students with the knowledge, skills, and ethical foundation to design intelligent systems that bridge societal gaps, improve quality of life, and create impactful solutions for regional, national, and global challenges.
- We strive to advance research, cultivate industry-academic collaborations, and promote the responsible and sustainable application of AI technologies to benefit humanity.

Program Specific Outcomes (PSO)

A graduate of the Artificial Intelligence & Machine Learning Program will demonstrate-

PSO1	An ability to apply the theoretical concepts and practical knowledge of Artificial Intelligence & Machine Learning in analysis, design, development and management of information processing systems and applications in the interdisciplinary domain.
PSO2	An ability to analyze a problem, and identify and define the computing infrastructure and operations requirements appropriate to its solution. AI & ML graduates should be able to work on large-scale computing systems.
PSO3	An understanding of professional, business and business processes, ethical, legal, security and social issues and responsibilities.
PSO4	Practice communication and decision-making skills through the use of appropriate technology and be ready for professional responsibilities.

NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS,PUNE
DEPARTMENT OF AIML ENGINEERING
ACADEMIC YEAR: 2025-26

PROGRAM OUTCOMES (POS)

Students are expected to know and be able to–

PO1	Engineering knowledge	An ability to apply knowledge of mathematics, computing, science, engineering and technology.
PO2	Problem analysis	An ability to define a problem and provide a systematic solution with the help of conducting experiments, analyzing the problem and interpreting the data.
PO3	Design / Development of Solutions	An ability to define a problem and provide a systematic solution with the help of conducting experiments, analyzing the problem and interpreting the data.
PO4	Conduct Investigations of Complex Problems	An ability to identify, formulates, and provides systematic solutions to complex engineering/Technology problems.
PO5	Modern Tool Usage	An ability to use the techniques, skills, and modern engineering technology tools, standard processes necessary for practice as a IT professional.
PO6	The Engineer and Society	An ability to apply mathematical foundations, algorithmic principles, and computer science theory in the modeling and design of computer-based systems with necessary constraints and assumptions.
PO7	Environment and Sustainability	An ability to analyze and provide solution for the local and global impact of information technology on individuals, organizations and society.
PO8	Ethics	An ability to understand professional, ethical, legal, security and social issues and responsibilities.
PO9	Individual and Team Work	An ability to function effectively as an individual or as a team member to accomplish a desired goal(s).
PO10	Communication Skills	An ability to engage in life-long learning and continuing professional development to cope up with fast changes in the technologies/tools with the help of electives, professional organizations and extra-curricular activities.

PO11	Project Management and Finance	An ability to communicate effectively in engineering community at large by means of effective presentations, report writing, paper publications, demonstrations.
PO12	Life-long Learning	An ability to understand engineering, management, financial aspects, performance, optimizations and time complexity necessary for professional practice.

Course Objectives:

1. To establish a strong conceptual foundation in database systems, covering fundamental principles, technologies, and best practices.
2. To understand the fundamental concepts of Relational Database Management System.
3. To introduce systematic approaches for database design, including Entity-Relationship modeling and normalization techniques.
4. To equip students with hands-on experience in SQL and procedural extensions (PL/SQL) for effective database interaction.
5. To familiarize with the key aspects of transaction processing, concurrency control and recovery management.
6. To learn and understand various database architectures and applications.
7. To introduce the recent trends in database technology.

Course Outcomes:

CO1: Explain the fundamental concepts, architecture, and functionalities of database management systems.

CO2: Analyze and design relational database (RDBMS) model to represent real-world database applications and demonstrate RDBMS principles.

CO3: Improve the database design through normalization.

CO4: Formulate database queries using SQL and PL/SQL for efficient data retrieval and manipulation.

CO5: Demonstrate ACID properties for transaction management and describe concurrency control protocols.

CO6: Explore and discuss recent trends in database technologies.

NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS,PUNE
DEPARTMENT OF AIML ENGINEERING
ACADEMIC YEAR: 2025-26

CO-PO-PSO Mapping for DBMS Practical

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CO1	2	1	1	1	2	-	-	-	-	-	-	-	3	1	1	-
CO2	3	3	2	2	3	-	-	-	1	-	1	1	3	3	1	1
CO3	3	2	3	2	2	-	1	-	-	-	-	-	2	2	2	-
CO4	3	3	3	3	3	-	-	-	2	1	2	2	3	3	1	2
CO5	2	3	2	3	2	1	2	2	2	-	1	1	2	3	3	1
CO6	1	1	1	1	2	-	1	1	1	2	1	-	2	2	3	3

NAVSAHYADRI

EDUCATION SOCIETY’S GROUP OF INSTITUTIONS,PUNE

DEPARTMENT OF AIML ENGINEERING

ACADEMIC YEAR: 2025-26

Experiment No. 1

Title

Study of MySQL Open Source Database Management System

Aim

To study MySQL open-source database software and understand its architecture, efficiency, scalability, and transactional properties.

Objectives

1. To understand MySQL architecture and components
2. To study efficiency and performance characteristics of MySQL
3. To understand scalability and concurrency support
4. To study transaction management and ACID compliance in MySQL

Course Outcomes Addressed

- **CO2:** Understand the relational database systems

PO & PSO Mapping

- **PO5:** Modern Tool Usage
- **PSO2:** Large-scale computing systems

Hardware Requirements

- Computer system with minimum **Intel i3 / AMD equivalent processor**
- Minimum **4 GB RAM**
- Minimum **10 GB free disk space**
- Internet connection (for installation and documentation access)

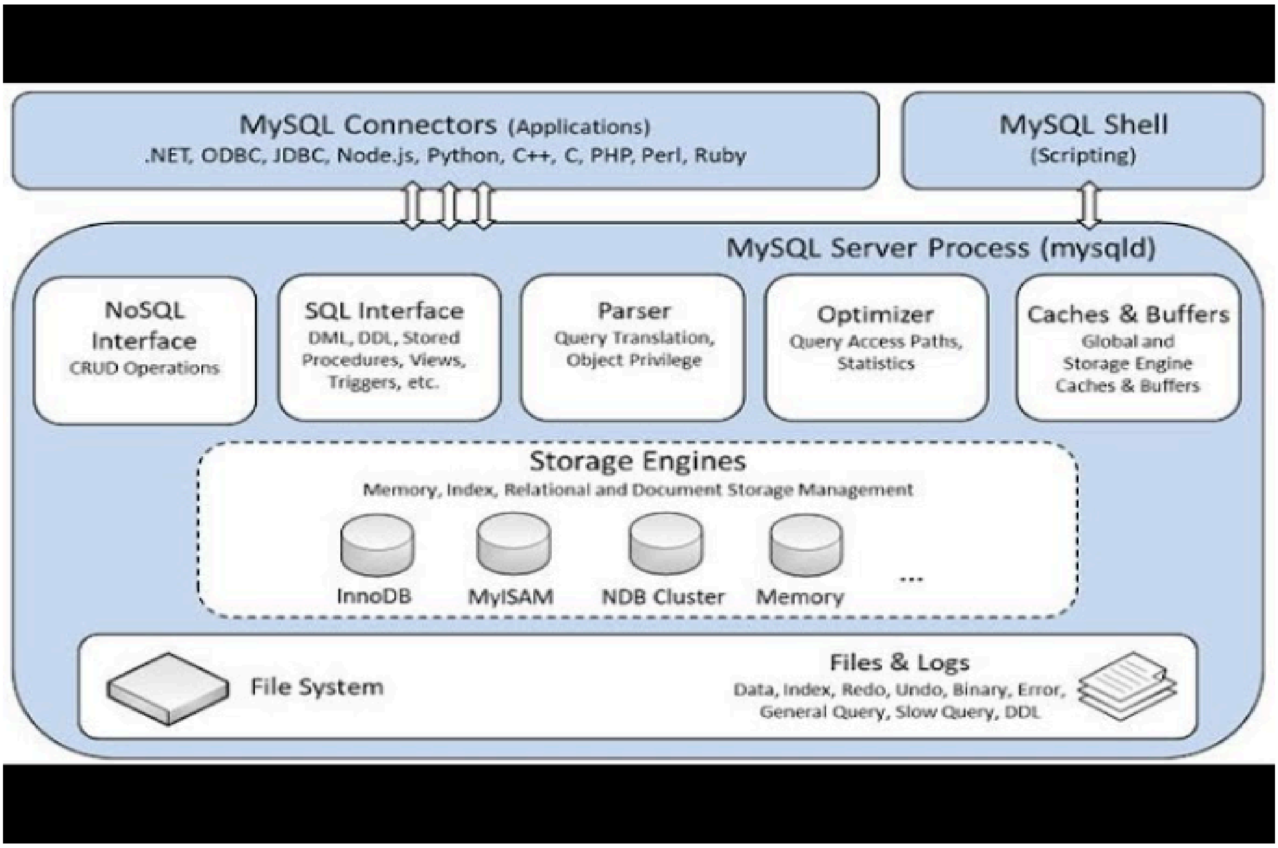
Software Requirements

- Operating System: **Windows / Linux**
- **MySQL Server (Community Edition)**
- **MySQL Workbench**
- Web Browser (Chrome / Firefox)

Theory

MySQL is a widely used **open-source Relational Database Management System (RDBMS)** developed and maintained by Oracle Corporation. It follows the **client–server architecture**, where the MySQL Server handles all database operations and clients communicate using SQL commands.

MySQL Architecture



MySQL architecture consists of:

- **Client Layer:** Applications and tools such as MySQL Workbench
- **SQL Layer:** SQL parser, optimizer, and query execution engine
- **Storage Engine Layer:** Handles data storage (InnoDB, MyISAM, etc.)

The **InnoDB storage engine** is the default engine in MySQL, which supports:

- Transactions
- Row-level locking
- Crash recovery
- ACID properties

Efficiency and Performance

MySQL provides high performance through:

- Query optimization
- Indexing (B-tree, hash indexes)
- Caching mechanisms
- Efficient memory management

Scalability

MySQL supports scalability through:

- Vertical scaling (hardware upgrades)
- Horizontal scaling (replication, sharding)
- Multi-user concurrent access

Transaction Management

MySQL supports **ACID properties**:

- **Atomicity:** Ensures all operations in a transaction are completed
- **Consistency:** Database remains in a valid state
- **Isolation:** Concurrent transactions do not interfere
- **Durability:** Committed data is not lost

Algorithm / Procedure

1. Install MySQL Server and MySQL Workbench
2. Start MySQL Server service
3. Connect to MySQL using Workbench
4. Execute basic SQL commands
5. Observe MySQL features and behavior
6. Study transaction support using commit and rollback

Case Study / Schema

Consider a **Student Database System** used by an educational institution to store student details, courses, and results with multiple users accessing data concurrently.

SQL Commands / Steps

```
-- Display MySQL version
SELECT VERSION();
```

```
-- Show available databases
SHOW DATABASES;

-- Create a sample database
CREATE DATABASE StudentDB;

-- Use the database
USE StudentDB;

-- Create a table
CREATE TABLE Student (
   PRN INT PRIMARY KEY,
   Name VARCHAR(50),
   Dept VARCHAR(20)
);

-- Start a transaction
START TRANSACTION;
INSERT INTO Student VALUES (101, 'Amit', 'AIML');
ROLLBACK;
```

Expected Output

- List of databases displayed
- StudentDB database created
- Student table created
- Rollback operation removes inserted record

(Students should attach screenshots of command execution)

Viva Questions

1. What is MySQL?
2. What is meant by open-source software?
3. Explain MySQL architecture.
4. What is a storage engine in MySQL?
5. What is InnoDB?
6. Explain ACID properties.
7. How does MySQL handle concurrency?
8. Difference between MySQL and SQLite?
9. What is indexing in MySQL?
10. Where is MySQL used in industry?

Observation

- MySQL supports multi-user access
- Transactions ensure data consistency
- Indexing improves query performance

Result / Conclusion

Thus, MySQL open-source database software was studied successfully. Students understood MySQL architecture, efficiency, scalability, transaction management, and its importance in real-world and industrial applications.

Experiment No. 2

Title

Study of SQLite Database System and Comparison with SQL-based Databases

Aim

To study SQLite database system, understand its features and applications, and compare SQLite with SQL-based relational databases such as MySQL.

Objectives

1. To understand the concept and architecture of SQLite
2. To study the uses and applications of SQLite
3. To compare SQLite with SQL-based databases like MySQL
4. To understand the advantages and limitations of SQLite

Course Outcomes Addressed

- **CO2:** Understand the relational database systems

PO & PSO Mapping

- **PO5:** Modern Tool Usage
- **PSO2:** Large-scale computing systems

Hardware Requirements

- Computer system with minimum **Intel i3 / AMD equivalent processor**
- Minimum **4 GB RAM**
- Minimum **5 GB free disk space**

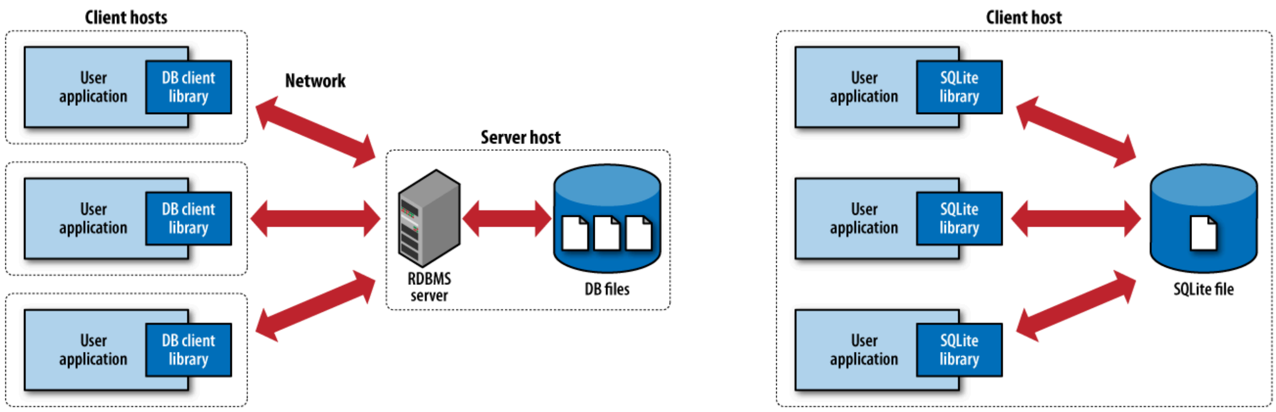
Software Requirements

- Operating System: **Windows / Linux**
- **SQLite Studio / DB Browser for SQLite**
- Web Browser (Chrome / Firefox)

Theory

SQLite is a **lightweight, serverless, self-contained relational database engine** that does not require a separate server process. Unlike traditional client–server databases such as MySQL or Oracle, SQLite stores the entire database in a **single file** on disk.

Architecture of SQLite



(a) Traditional client-server architecture (b) SQLite serverless architecture

SQLite follows a **serverless architecture**:

- No separate database server
- Direct access to the database file
- SQL engine embedded within the application

Uses of SQLite

SQLite is widely used in:

- Mobile applications (Android, iOS)
- Embedded systems
- Desktop applications
- Web browsers
- IoT devices
- Prototyping and testing environments

Advantages of SQLite

- Zero configuration
- Lightweight and fast
- Easy deployment
- ACID compliant
- Portable database files

Limitations of SQLite

- Limited support for high concurrency
- Not suitable for large-scale distributed systems
- Limited scalability compared to MySQL

Comparison: SQLite vs SQL-based Databases (MySQL)

Feature	SQLite	MySQL
Architecture	Serverless	Client–Server
Configuration	Zero	Required
Storage	Single file	Multiple files
Scalability	Limited	High
Concurrency	Limited	High
Use Case	Mobile, embedded	Web, enterprise

Algorithm / Procedure

1. Install SQLite Studio or DB Browser for SQLite
2. Create a new SQLite database file
3. Create tables using SQL commands
4. Insert and retrieve records
5. Observe SQLite features and limitations
6. Compare SQLite behavior with MySQL

Sample Case Study / Schema

Consider a **Mobile Notes Application** that stores user notes locally on the device using SQLite for fast access and offline functionality.

SQL Commands / Steps

```
-- Check SQLite version
SELECT sqlite_version();

-- Create table
CREATE TABLE Notes (
    NoteID INTEGER PRIMARY KEY,
    Title TEXT,
    Content TEXT
);

-- Insert records
INSERT INTO Notes VALUES (1, 'DBMS', 'SQLite is lightweight');
-- Retrieve data
SELECT * FROM Notes;
```

Expected Output

- SQLite version displayed
- Notes table created
- Record inserted successfully
- Notes data retrieved

(Students should attach screenshots of execution)

Viva Questions

1. What is SQLite?
2. What is meant by serverless database?
3. Where is SQLite used?
4. How is SQLite different from MySQL?
5. What are the advantages of SQLite?
6. Is SQLite ACID compliant?
7. Why is SQLite preferred in mobile applications?
8. Can SQLite handle multiple users?
9. What are limitations of SQLite?
10. When should SQLite not be used?

Observation

- SQLite does not require a server
- Database stored in a single file
- Suitable for small-scale applications

Result / Conclusion

Thus, SQLite database system was studied successfully. Students understood SQLite architecture, applications, advantages, limitations, and its comparison with SQL-based databases such as MySQL.

Experiment No. 3

Title
Database Design using ER / EER Diagram for a Real-World System

Aim
To design a database with at least two entities and relationships between them using ER / EER modeling techniques.

Objectives

- To identify entities and attributes from a problem domain
- To define relationships and mapping cardinalities
- To apply ER / EER modeling concepts
- To design a conceptual database model

Course Outcomes Addressed
CO1: Analyze database models and entity–relationship models
CO3: Design and implement a database schema for a given problem domain

PO & PSO Mapping
PO3: Design / Development of Solutions
PSO2: Large-scale computing systems

Hardware Requirements

- Computer system with minimum Intel i3 / AMD equivalent processor
- Minimum 4 GB RAM
- Drawing sheet / Computer system for diagram preparation

Software Requirements

- Operating System: Windows / Linux
- MySQL Workbench / Draw.io / Lucidchart / MS Visio / ERDPlus
- Web Browser (optional)

Theory
Database design is a critical phase in the development of any database system. The Entity–Relationship (ER) model is a conceptual data model used to represent real-world objects and their relationships in a structured form.

An **entity** represents a real-world object such as Student, Course, or Department.
An **attribute** describes properties of an entity, such as Name, ID, or Age.
A **relationship** represents how entities are connected to each other.

The **Extended Entity–Relationship (EER)** model enhances the ER model by including:

- Specialization
- Generalization
- Inheritance
- Weak entities

ER/EER diagrams help in:

- Clear visualization of database structure
- Reducing data redundancy
- Improving database integrity
- Serving as a blueprint for logical design

Problem Statement / Case Study

Design a database for a **Student–Course Registration System** where:

- Each student can enroll in multiple courses
- Each course can have multiple students

Identified Entities and Attributes
Entity 1: STUDENT

- StudentID (Primary Key)
- Name
- Department
- Email

Entity 2: COURSE

- CourseID (Primary Key)
- CourseName
- Credits

Relationship: ENROLLS

- Relationship between STUDENT and COURSE
- Cardinality: **Many-to-Many (M:N)**

ER / EER Diagram Description

- STUDENT entity connected to COURSE entity through ENROLLS relationship
- Primary keys identified (underlined)
- Mapping cardinalities specified (M:N)

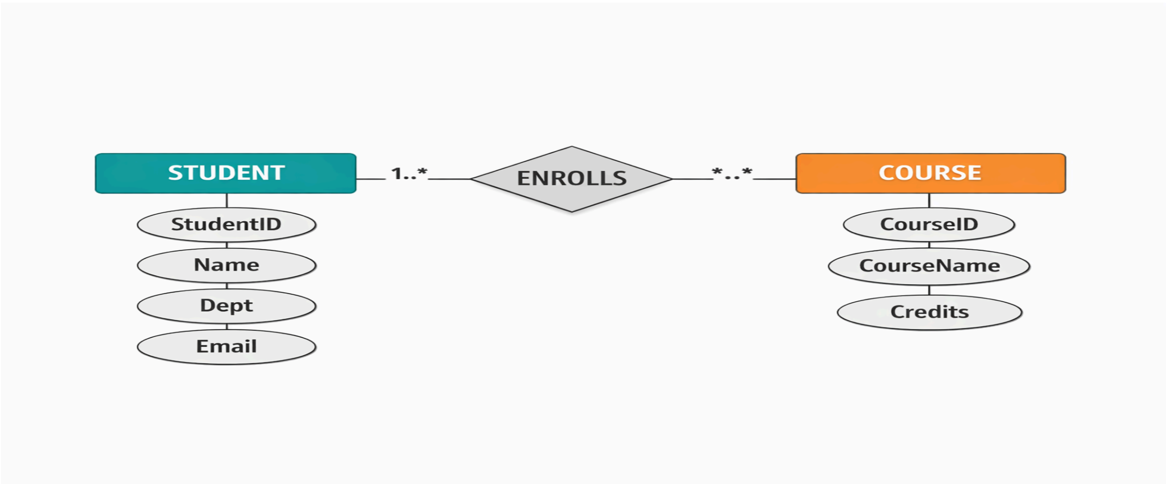
- Participation constraints defined

(Students should draw a neat ER/EER diagram manually or using a tool and attach it in the lab manual)

Algorithm / Procedure

1. Read and analyze the problem statement
2. Identify entities (e.g., STUDENT, COURSE) and list attributes
3. Determine primary keys for each entity
4. Identify relationships (ENROLLS) and cardinalities (M:N)
5. Draw ER diagram using tool (Draw.io/ERDPlus)
6. Add keys (underline PK), participation symbols (double line)
7. Validate: Check for redundancy/completeness
8. Convert ER to relational tables (optional step)

Sample ER Diagram Representation (Textual)



Relational Schema Conversion

STUDENT (StudentID PK, Name, Dept, Email)

COURSE (CourseID PK, CourseName, Credits)

ENROLLMENT (StudentID FK, CourseID FK) // Resolves M:N

Expected Output

- Neat ER diagram with:
  - Rectangles (entities), Ovals (attributes)
  - Underlined PKs, Diamond (relationship)
  - Cardinality symbols (crow's foot/1:M notation)
  - Participation constraints (double/single lines)

- Attach: Tool screenshot + hand-drawn diagram scan

Viva Questions

1. What is an entity?
2. What is an attribute?
3. What is a relationship?
4. What is cardinality?
5. Difference between ER and EER model?
6. What is a weak entity? Give example.
7. What is generalization?
8. What is specialization?
9. Why ER modeling is important?
10. How ER diagram helps in database design?
11. What is weak entity? Give example.
12. How to convert M:N relationship to tables?

Observation

- ER model simplifies database design
- Relationships help define data associations
- Cardinality ensures data consistency

Result / Conclusion

Thus, a database system was successfully designed using ER / EER modeling concepts. Entities, attributes, relationships, and constraints were identified and represented using an ER diagram.

Experiment No. 4

Title

Database Implementation using SQL: Creation of Tables with Primary Key, Foreign Key and Constraints

Aim

To implement a relational database by creating tables using SQL with primary key, foreign key, and integrity constraints, ensuring data accuracy, consistency, and reliability.

Objectives

- To convert ER diagram into relational tables
- To understand the role of primary key and foreign key
- To apply domain, entity, and referential integrity constraints
- To implement database schema using SQL DDL commands

Course Outcomes Addressed

CO3: Design and implement a database schema for a given problem domain
CO4: Populate and query a database using SQL DDL and DML commands

PO & PSO Mapping

PO3: Design / Development of Solutions
PO5: Modern Tool Usage
PSO2: Large-scale computing systems

Hardware Requirements

- Intel i3 / AMD equivalent processor
- Minimum 4 GB RAM
- Minimum 10 GB hard disk space

Software Requirements

- Operating System: Windows / Linux
- MySQL Server (Community Edition)
- MySQL Workbench

Theory

After completing ER modeling, the next phase of database development is logical and physical implementation. This phase converts conceptual design into actual database tables using SQL.

1. Relational Database Tables

A table is a collection of related data organized into rows and columns. Each table represents an entity from the ER diagram.

Example:

- STUDENT table represents Student entity
- COURSE table represents Course entity

2. Primary Key (PK)

A Primary Key uniquely identifies each record in a table.

Characteristics

- Must be unique
- Cannot contain NULL values
- Only one primary key per table
- Ensures entity integrity

Example

StudentID INT PRIMARY KEY

Application

- Roll Number in college system
- Account Number in bank
- Aadhaar Number in government database

3. Foreign Key (FK)

A Foreign Key establishes a relationship between two tables by referencing the primary key of another table.

Purpose

- Maintains referential integrity
- Prevents orphan records
- Enforces relationship rules

Example

FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID)

Application

- OrderID in ORDER_DETAILS table
- CustomerID in SALES table
- StudentID in ENROLLMENT table

4. NOT NULL Constraint

Ensures that a column cannot store NULL values.

Example

Name VARCHAR(50) NOT NULL

Application

- Student name cannot be empty
- Product name must be provided

5. UNIQUE Constraint

Ensures that values in a column are unique.

Example

Email VARCHAR(50) UNIQUE

Application

- Email ID in user database
- Username in login system

6. CHECK Constraint

Validates data based on a specified condition.

Example

Credits INT CHECK (Credits >= 1)

Application

- Age must be ≥ 18
- Salary must be positive
- Marks between 0 and 100

7. Composite Primary Key

When more than one column together uniquely identifies a record.

Example

PRIMARY KEY (StudentID, CourseID)

Application

- Student–Course registration
- Order–Product mapping

8. Referential Actions (CASCADE)

Controls behavior when referenced data is updated or deleted.

ON DELETE CASCADE

- **Deletes dependent records automatically**

ON UPDATE CASCADE

- **Updates foreign key values automatically**

Application

- Deleting a student removes enrollment records
- Updating course ID updates enrollment table

Problem Statement / Case Study

Implement the Student–Course Registration System database designed in Experiment–3 using SQL commands and constraints.

Relational Schema

STUDENT (StudentID PK, Name, Dept, Email)

COURSE (CourseID PK, CourseName, Credits)

ENROLLMENT (StudentID FK, CourseID FK)

Algorithm / Procedure

1. Start MySQL server and open MySQL Workbench
2. CREATE DATABASE CollegeDB;
3. USE CollegeDB;
4. CREATE TABLE STUDENT with PK, NOT NULL, UNIQUE
5. CREATE TABLE COURSE with PK, CHECK constraint
6. CREATE TABLE ENROLLMENT with composite PK, FKs, CASCADE
7. Verify: DESCRIBE each table
8. Test constraints: INSERT violating data
9. Screenshot outputs

SQL Commands / Implementation

sql

```
CREATE DATABASE CollegeDB;

USE CollegeDB;

CREATE TABLE STUDENT (

    StudentID INT PRIMARY KEY,

    Name VARCHAR(50) NOT NULL,

    Dept VARCHAR(20) NOT NULL,

    Email VARCHAR(50) UNIQUE

);

CREATE TABLE COURSE (

    CourseID INT PRIMARY KEY,

    CourseName VARCHAR(50) NOT NULL,

    Credits INT CHECK (Credits >= 1)

);

CREATE TABLE ENROLLMENT (

    StudentID INT,

    CourseID INT,

    PRIMARY KEY (StudentID, CourseID),
```

```
    FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID)

        ON DELETE CASCADE ON UPDATE CASCADE,

    FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)

        ON DELETE CASCADE ON UPDATE CASCADE

);
```

Verification Commands

sql

```
DESCRIBE STUDENT;

DESCRIBE ENROLLMENT;

SHOW CREATE TABLE ENROLLMENT;
```

Test Cases

sql

```
-- Success

INSERT INTO STUDENT VALUES (101, 'Amit', 'AIML', 'amit@email.com');

INSERT INTO COURSE VALUES (501, 'DBMS', 4);

-- Fail: Duplicate PK

INSERT INTO STUDENT VALUES (101, 'Raj', 'CSE', 'raj@email.com');  -- ERROR!

-- Fail: NULL Name

INSERT INTO STUDENT VALUES (102, NULL, 'CSE', 'test@email.com');  -- ERROR!
```

Expected Output

- Screenshots of:
  - DESCRIBE output (PK/FK visible)
  - Error messages (constraint violations)
  - SHOW CREATE TABLE (FK definition)

Real-World Applications

- College: Student-course enrollment
- E-commerce: Orders-products

- Banking: Customer-account
- Hospital: Patient-doctor

Viva Voce Questions

1. Why primary key is important?
2. What happens if primary key is duplicated?
3. Difference between PK and FK?
4. What is referential integrity?
5. Explain composite key with example.
6. What is ON DELETE CASCADE?
7. Can foreign key reference non-primary key?
8. Why constraints are important in DBMS?
9. What happens if CHECK condition fails?
10. How constraints improve data security?
11. What is InnoDB vs MyISAM for FK support?
12. Explain CASCADE vs RESTRICT in FK actions.

Observation

- Data integrity maintained
- Redundancy reduced
- Database structure robust

Result / Conclusion

Thus, a relational database was successfully implemented using SQL with primary key, foreign key, and integrity constraints. This ensures accuracy, consistency, and reliability of data in real-world applications.

Experiment No. 5

Title

SQL Operations on Database Tables using DDL, DML and Advanced Queries

Aim

To perform various SQL operations on database tables using **DDL and DML commands**, including ALTER, DROP, INDEX, relational operators, pattern matching, aggregate functions, nested queries, set operators, views, and sorting.

Objectives

1. To modify database tables using ALTER and DROP commands
2. To improve query performance using INDEX operations
3. To retrieve data using relational operators and pattern matching
4. To analyze data using aggregate functions, GROUP BY and HAVING
5. To implement nested queries and set operators
6. To create and use views for data abstraction
7. To sort query results effectively

Course Outcomes Addressed

- **CO4:** Populate and query a database using SQL DDL and DML commands

PO & PSO Mapping

- **PO3:** Design / Development of Solutions
- **PO5:** Modern Tool Usage
- **PSO2:** Large-scale computing systems

Hardware Requirements

- Intel i3 / AMD equivalent processor
- Minimum **4 GB RAM**
- Minimum **10 GB free disk space**

Software Requirements

- Operating System: **Windows / Linux**
- **MySQL Server (Community Edition)**
- **MySQL Workbench**

Theory

After creating tables with constraints, SQL provides various commands to **modify structure, retrieve meaningful information, and analyze data efficiently**.

a. ALTER TABLE

The **ALTER TABLE** command is used to modify the structure of an existing table.

Operations

- Add a new column
- Modify column data type
- Drop a column

Example

ALTER TABLE STUDENT ADD Phone VARCHAR(10);

Application

- Adding mobile number in student database
- Adding GST number in customer table

b. DROP TABLE

The **DROP TABLE** command permanently deletes a table and its data.

Example

DROP TABLE TEMP_TABLE;

Application

- Removing obsolete tables
- Cleaning temporary or test data

c. INDEX Operations

Indexes improve the **speed of data retrieval**.

Types

- Single-column index
- Composite index

Example CREATE INDEX idx_name ON STUDENT(Name);

Application

- Fast searching in large databases
- Used in banking, e-commerce systems

d. Relational Operators

Used in WHERE clause to filter records.

Operator	Meaning
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal

Example
SELECT * FROM STUDENT WHERE StudentID > 100;

Application

- Filtering employees by salary
- Selecting students above cutoff marks

e. Pattern Matching

Pattern matching is done using **LIKE** operator.

Symbol	Meaning
%	Any number of characters
_	Single character

Example

```
SELECT * FROM STUDENT WHERE Name LIKE 'A%';
```

Application

- Search by name
- Auto-suggestion systems

f. Aggregate Functions with GROUP BY & HAVING

Aggregate functions perform calculations on data.

Function	**Purpose**
COUNT	Number of rows
SUM	Total
AVG	Average
MIN	Minimum
MAX	Maximum

Example

```
SELECT Dept, AVG(Marks)
FROM RESULT
GROUP BY Dept
HAVING AVG(Marks) > 60;
```

Application

- Department-wise result analysis
- Sales analysis by region

g. Nested Queries (Subqueries)

A query inside another query.

Example

SELECT Name FROM STUDENT

WHERE Marks > (SELECT AVG(Marks) FROM RESULT);

Application

- Finding top performers
- Identifying employees earning above average

h. Set Operators

Combine results of two queries.

Operator	Purpose
UNION	Combine without duplicates
UNION ALL	Combine with duplicates
INTERSECT	Common records
EXCEPT	Difference

Example

SELECT StudentID FROM STUDENT_A

UNION

SELECT StudentID FROM STUDENT_B;

Application

- Comparing two semesters' student lists
- Merging data from multiple branches

i. Views

A **VIEW** is a virtual table based on a query.

Example

CREATE VIEW FailedStudents AS

SELECT Name FROM RESULT WHERE Marks < 40;

Application

- Data security
- Simplifying complex queries
- Role-based access

j. Sorting

Sorting is done using **ORDER BY** clause.

Example

SELECT * FROM STUDENT ORDER BY Name ASC;

Application

- Ranking students
- Sorting employees by salary

Algorithm / Procedure

1. Open MySQL Workbench
2. Select the database
3. Perform ALTER operations
4. Create and drop indexes
5. Execute relational queries
6. Apply pattern matching
7. Use aggregate functions
8. Execute nested queries
9. Create and query views
10. Sort results

Expected Output

- Modified table structure
- Query results displayed correctly

- Improved performance using indexes

(Attach screenshots of outputs)

Real-World Applications

1. **University ERP** – student filtering, ranking
2. **Banking System** – customer transaction analysis
3. **E-commerce** – sales reports and customer behavior
4. **HR Management** – employee performance analysis
5. **Healthcare Systems** – patient data retrieval

Viva Questions

1. Difference between ALTER and UPDATE?
2. Why indexing improves performance?
3. What is difference between WHERE and HAVING?
4. What is view? Is data stored in view?
5. Explain nested query with example
6. Difference between UNION and UNION ALL?
7. Why ORDER BY is important?
8. When should DROP TABLE be avoided?
9. What is pattern matching?
10. Where are aggregate functions used in real life?

Observation

- SQL operations enhance data manipulation
- Complex data retrieval becomes easier
- Performance improves using indexes

Result / Conclusion

Thus, various SQL operations were successfully performed on database tables. Students learned how to modify tables, retrieve meaningful information, analyze data, and apply SQL concepts in real-world applications.

Experiment No. 6

Title

Implementation of PL/SQL Stored Procedure and Stored Function

Aim

To write, execute, and demonstrate the use of **PL/SQL stored procedures and stored functions** to perform operations on a database.

Objectives

1. To understand the concept of PL/SQL
2. To write and execute a stored procedure
3. To write and execute a stored function
4. To understand the difference between procedure and function
5. To demonstrate reusability and modularity in database programming

Course Outcomes Addressed

- **CO5:** Implement PL/SQL including stored procedures, stored functions, and triggers

PO & PSO Mapping

- **PO3:** Design / Development of Solutions
- **PO5:** Modern Tool Usage
- **PSO2:** Large-scale computing systems

Hardware Requirements

- Computer system with **Intel i3 / AMD equivalent processor**
- Minimum **4 GB RAM**
- Minimum **10 GB free disk space**

Software Requirements

- Operating System: **Windows / Linux**
- **Oracle XE / MySQL (with procedural extensions)**
- **SQL*Plus / MySQL Workbench**

Theory

1. PL/SQL Overview

PL/SQL (Procedural Language/SQL) is a **block-structured, procedural extension of SQL** that allows developers to combine SQL statements with procedural constructs such as loops, conditions, and exception handling.

PL/SQL improves database programming by:

- Reducing network traffic
- Improving performance
- Providing modular and reusable code

2. Stored Procedure

A **stored procedure** is a named PL/SQL block stored permanently in the database that performs a specific task.

Characteristics

- Can accept input and output parameters
- Does **not return a value directly**
- Can contain SQL and PL/SQL statements
- Improves performance by pre-compiling

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name

AS

BEGIN

   -- statements

END;
```

3. Stored Function

A **stored function** is similar to a procedure but **returns a single value**.

Characteristics

- Must return a value using RETURN statement
- Can be used in SQL queries
- Improves modularity and code reuse

Syntax

```
CREATE OR REPLACE FUNCTION function_name

RETURN datatype

AS

BEGIN

   RETURN value;

END;
```

4. Difference between Procedure and Function

Procedure	Function
Does not return value	Returns value
Called using EXEC	Used in expressions
Used for actions	Used for calculations

Problem Statement / Case Study

Implement PL/SQL programs for a College Management System to:

- Calculate total salary of employees using a procedure
- Calculate annual salary using a function

Database Table Used

EMP (EmpID, EmpName, Salary, Dept)

Algorithm / Procedure

A. Stored Procedure

1. Create a procedure to calculate and display total salary of employees in a department
2. Execute the procedure

B. Stored Function

1. Create a function to calculate annual salary
2. Call the function inside a SELECT statement

PL/SQL Implementation

1. Stored Procedure

```
CREATE OR REPLACE PROCEDURE total_salary_dept (
    d_name IN VARCHAR2
)
AS
    total_sal NUMBER;
BEGIN
    SELECT SUM(Salary)
    INTO total_sal
    FROM EMP
    WHERE Dept = d_name;

    DBMS_OUTPUT.PUT_LINE('Total Salary of ' || d_name || ' Department = ' ||
total_sal);
END;
/
```

Execution

```
EXEC total_salary_dept('AIML');
```

2. Stored Function

```
CREATE OR REPLACE FUNCTION annual_salary (
    emp_sal IN NUMBER
```

```
)

RETURN NUMBER

AS

BEGIN

   RETURN emp_sal * 12;

END;

/
```

Execution

```
SELECT EmpName, annual_salary(Salary) AS Annual_Salary

FROM EMP;
```

Expected Output

- Procedure displays department-wise total salary
- Function returns annual salary for each employee

(Attach screenshots of execution output)

Real-World Applications

1. **Payroll Systems** – salary calculation
2. **Banking Systems** – interest calculation
3. **University ERP** – fee computation
4. **E-commerce** – discount and tax calculation
5. **Healthcare Systems** – billing operations

Viva Questions

1. What is PL/SQL?
2. Why PL/SQL is faster than SQL?
3. What is stored procedure?
4. What is stored function?
5. Difference between procedure and function?
6. Can a function be called from SELECT statement?
7. What are advantages of stored procedures?
8. What is modularity?
9. What is DBMS_OUTPUT?

10. Where are procedures used in industry?

Observation

- Procedures improve code reusability
- Functions simplify complex calculations
- PL/SQL improves database performance

Result / Conclusion

Thus, PL/SQL stored procedures and stored functions were successfully implemented and executed. Students understood procedural database programming, modular design, and real-world applications of PL/SQL.

Experiment No. 7

Title

Implementation of Database Triggers using PL/SQL

Aim

To write, execute, and demonstrate the use of **database triggers** to automatically enforce business rules and maintain data integrity in a database.

Objectives

1. To understand the concept and need of database triggers
2. To study different types of triggers
3. To write and execute BEFORE and AFTER triggers
4. To use triggers for auditing and validation
5. To understand real-world applications of triggers

Course Outcomes Addressed

- **CO5:** Implement PL/SQL including stored procedures, stored functions, and triggers

PO & PSO Mapping

- **PO3:** Design / Development of Solutions
- **PO5:** Modern Tool Usage
- **PSO2:** Large-scale computing systems

Hardware Requirements

- Computer system with **Intel i3 / AMD equivalent processor**
- Minimum **4 GB RAM**
- Minimum **10 GB free disk space**

Software Requirements

- Operating System: **Windows / Linux**
- **Oracle XE / MySQL (with trigger support)**
- **SQL*Plus / MySQL Workbench**

Theory

1. Introduction to Database Triggers

A **database trigger** is a special PL/SQL program that is **automatically executed (fired)** when a specific database event occurs on a table or view.

Triggers are **event-driven**, unlike procedures or functions which are explicitly called.

2. Need for Triggers

Triggers are used to:

- Enforce complex business rules
- Maintain data consistency automatically
- Perform auditing and logging
- Prevent invalid data entry
- Reduce dependency on application code

3. Triggering Events

Triggers are fired on:

- **INSERT**
- **UPDATE**
- **DELETE**

4. Timing of Triggers

- **BEFORE Trigger** – Executes before the DML operation
- **AFTER Trigger** – Executes after the DML operation

5. Types of Triggers

a. Row-Level Trigger

- Executes once for each affected row
- Uses :OLD and :NEW references

b. Statement-Level Trigger

- Executes once per SQL statement
- Does not use :OLD or :NEW

6. OLD and NEW References

- :OLD → Previous value of the row
- :NEW → New value of the row

Used mainly in **UPDATE** and **DELETE** triggers.

7. Advantages of Triggers

- Automatic execution
- Improved security
- Centralized business logic
- Reduced application complexity

Problem Statement / Case Study

Implement database triggers for an **Employee Management System** to:

1. Store old salary values before update
2. Log insert operations on STUDENT table

Database Tables Used

EMP (EmpID, EmpName, Salary, Dept)

EMP_AUDIT (EmpID, OldSalary, ChangeDate)

STUDENT (StudentID, Name, Dept)

Algorithm / Procedure

A. BEFORE UPDATE Trigger

1. Create an audit table
2. Write trigger to capture old salary
3. Execute UPDATE command
4. Verify audit record

B. AFTER INSERT Trigger

1. Write trigger on STUDENT table
2. Insert new record
3. Verify trigger execution

PL/SQL Trigger Implementation

1. BEFORE UPDATE Trigger (Salary Audit)

```
CREATE TABLE EMP_AUDIT (

    EmpID NUMBER,

    OldSalary NUMBER,

    ChangeDate DATE

);

CREATE OR REPLACE TRIGGER trg_before_update_salary

BEFORE UPDATE OF Salary ON EMP

FOR EACH ROW

BEGIN

    INSERT INTO EMP_AUDIT

    VALUES (:OLD.EmpID, :OLD.Salary, SYSDATE);

END;

/
```

Execution

```
UPDATE EMP SET Salary = Salary + 5000 WHERE EmpID = 101;
```

2. AFTER INSERT Trigger (Student Log)

```
CREATE OR REPLACE TRIGGER trg_after_insert_student

AFTER INSERT ON STUDENT

FOR EACH ROW

BEGIN

    DBMS_OUTPUT.PUT_LINE('New student record inserted');

END;
```

/

Execution

INSERT INTO STUDENT VALUES (201, 'Ravi', 'AIML');

Expected Output

- Old salary values stored in EMP_AUDIT table
- Trigger message displayed on student insertion

(Students should attach screenshots of trigger creation and output)

Real-World Applications

1. **Payroll Systems** – salary change audit
2. **Banking Systems** – transaction logging
3. **University ERP** – student admission tracking
4. **E-commerce Systems** – order status updates
5. **Healthcare Systems** – patient record audit

Viva Questions

1. What is a trigger?
2. Difference between trigger and procedure?
3. What is BEFORE trigger?
4. What is AFTER trigger?
5. What is row-level trigger?
6. What is :OLD and :NEW?
7. Can triggers be disabled?
8. Where are triggers used in industry?
9. What are disadvantages of triggers?
10. Can triggers call procedures?

Observation

- Triggers execute automatically
- Audit information is maintained
- Data integrity is ensured

Result / Conclusion

Thus, database triggers were successfully written and executed. Students learned how triggers enforce business rules, maintain audit logs, and improve data integrity in real-world database applications.